

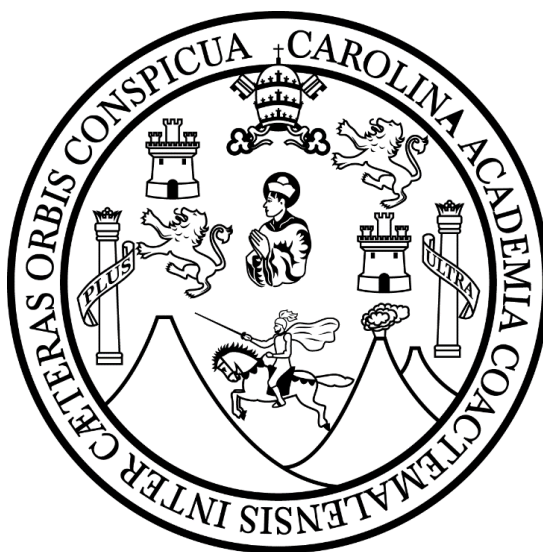
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ESCUELA DE CIENCIAS Y SISTEMAS

INTELIGENCIA ARTIFICIAL

SECCIÓN A



Manual Tecnico

Proyecto Fase 1 Grupo 9	
201504464	Herberth Abisai Avila Ruiz
202000562	Pedro Luis Pu Tavico
201602983	Jaime Alejandro Armira Us

Tabla de Contenido

Introducción.....	3
Tecnologías y Herramientas Utilizadas.....	3
Estructura del Proyecto.....	3
Implementación del Modelo.....	3
Cómo Ejecutar el Proyecto.....	3
Documentación de Código.....	4
Mantenimiento y Mejora.....	4

Introducción

Este proyecto consiste en un modelo de inteligencia artificial desarrollado con JavaScript, que permite a los usuarios interactuar mediante entradas en español. Utiliza una biblioteca específica para la creación de modelos de IA en JavaScript y Python.

Tecnologías y Herramientas Utilizadas

- **Lenguaje de Programación:** JavaScript (Ejecutado en el navegador) y Python.
- **Bibliotecas:**
 - TensorFlow
 - Torch
 - NLKT
 - HTML/CSS para la interfaz de usuario.
- **Desarrollo Web:** Utilización de HTML5, CSS3 y JavaScript para la construcción de la interfaz gráfica y la lógica de interacción.

Estructura del Proyecto

La estructura del proyecto está organizada en varias carpetas y archivos que manejan diferentes aspectos del chatbot:

- **Fase1/index.html:** Archivo HTML que define la estructura de la interfaz de usuario, donde el usuario puede interactuar con el chatbot.
- **Fase1/style.css:** Archivo para la personalización de la apariencia de la interfaz.
- **Fase1/chatbot.js:** Script principal para manejar la interacción entre el modelo ONNX y la interfaz de usuario. Carga el modelo ONNX, preprocesa las entradas y ejecuta la inferencia para obtener la respuesta del chatbot.
- **Fase1/modelo/chatstospot.py:** Contiene la lógica del modelo de red neuronal que procesa las entradas del usuario. Utiliza PyTorch para entrenar un modelo de clasificación de intenciones. - Se encarga de tokenizar, stemizar y construir la "bag of words" a partir de las entradas del usuario. - Define el modelo y entrena la red neuronal usando un conjunto de patrones de entrada.
- **Fase1/modelo/intents.json:** Archivo JSON que almacena los intents (intenciones) y patrones de entrada, junto con las respuestas posibles. - Cada intent tiene un "tag" que representa la categoría de la consulta del

usuario, un conjunto de "patterns" que son frases típicas que corresponden a ese intent, y las "responses" posibles que el chatbot puede dar.

- **Fase1/modelo/model.onnx:** Modelo exportado en formato ONNX, que es una versión optimizada del modelo de red neuronal. Este formato facilita su integración en la web usando ONNX.js.
- **Fase1/modelo/redNeuronal.py:** Define la estructura de la red neuronal, que incluye tres capas completamente conectadas (input, hidden y output). Utiliza la función de activación ReLU entre las capas.
- **Fase1/modelo/utilidades.py:** Contiene funciones auxiliares, como `tokenize` para dividir las frases en palabras, `stem` para reducir las palabras a su raíz, y `bag\_of\_words` para crear una representación vectorial de las entradas del usuario.

Implementación del Modelo

El modelo se implementa utilizando PyTorch para el entrenamiento de la red neuronal y luego se exporta a ONNX para que pueda ser utilizado en la web. A continuación, se describen los pasos principales:

- **Entrenamiento del modelo:**
 - Se utiliza el archivo `intents.json` como fuente de datos para entrenar el modelo de clasificación de intenciones.
 - Cada frase en los "patterns" se tokeniza, se stemiza y luego se transforma en una "bag of words". La red neuronal se entrena para predecir la categoría (intent) correspondiente.
- **Exportación del modelo a ONNX:**
 - Una vez entrenado el modelo en PyTorch, se exporta al formato ONNX usando la función `torch.onnx.export()`. Esto facilita su uso en aplicaciones web, ya que ONNX es compatible con JavaScript a través de ONNX.js.
- **Interacción con la web:**
 - El archivo `chatbot.js` carga el modelo ONNX en el navegador y procesa las entradas del usuario. El modelo predice la intención del usuario y devuelve una respuesta basada en el intent identificado.
 - La función `preprocessInput()` convierte la entrada del usuario en un formato compatible con el modelo, mientras que `runInference()` ejecuta la inferencia usando ONNX.js.

Cómo Ejecutar el Proyecto

1. Clona el repositorio desde GitHub.
2. Abre el archivo **index.html** en cualquier navegador web.
3. El modelo estará listo para interactuar con las entradas del usuario.

Documentación de Código

- **Funciones Principales:**
 - **tokenize(sentence):** Convierte una frase en una lista de palabras/tokens. Utiliza la biblioteca `nltk`.

```
# modelo/utilidades.py

def tokenize(sentence):
    """
    Convierte una frase en una lista de palabras o tokens.
    """
    return nltk.word_tokenize(sentence)

def stem(word):
    """
    Aplica stemming, reduciendo las palabras a su raíz.
    """
    return stemmer.stem(word.lower())

def bag_of_words(tokenized_sentence, words):
    """
```

```

    Crea una representación de la frase en forma de 'bag
of words',

    donde 1 representa la presencia de una palabra y 0 su
ausencia.

    """

    sentence_words = [stem(word) for word in
tokenized_sentence]

    bag = np.zeros(len(words), dtype=np.float32)

    for idx, w in enumerate(words):

        if w in sentence_words:

            bag[idx] = 1

    return bag

```

- **stem(word):** Aplica stemming (reducción de palabras a su raíz) usando `PorterStemmer`. Por ejemplo, "organize", "organizes", y "organizing" se reducen a "organ".
- **bag_of_words(tokenized_sentence, words):** Crea un vector de palabras en el que cada posición indica si la palabra está presente en la frase ingresada por el usuario.
- **Clase RedNeuronal:** Define una red neuronal simple con tres capas. Utiliza la función de activación ReLU y la pérdida CrossEntropyLoss para la optimización durante el entrenamiento.

```

# modelo/redNeuronal.py

import torch

import torch.nn as nn

class RedNeuronal(nn.Module):

    def __init__(self, input_size, hidden_size,
num_classes):

        super(RedNeuronal, self).__init__()

```

```

        self.l1 = nn.Linear(input_size, hidden_size)

        self.l2 = nn.Linear(hidden_size, hidden_size)

        self.l3 = nn.Linear(hidden_size, num_classes)

        self.relu = nn.ReLU()

    def forward(self, x):

        out = self.l1(x)

        out = self.relu(out)

        out = self.l2(out)

        out = self.relu(out)

        out = self.l3(out)

        return out

```

- **Función `runInference()` en chatbot.js:** Realiza la inferencia del modelo cargado en el navegador y obtiene el vector de resultados.

```

// Fase1/chatbot.js

async function loadModel() {

    const modelUrl = './modelo/model.onnx';

    const session = await
ort.InferenceSession.create(modelUrl);

    console.log('Model loaded');

    return session;

}

async function runInference(session, inputData) {

    const inputTensor = new ort.Tensor('float32',
inputData, [1, inputData.length]);

```

```
const feeds = { input: inputTensor };

const results = await session.run(feeds);

const outputName = session.outputNames[0];

return results[outputName].data;
}
```

- **Función `getChatbotResponse()` en chatbot.js:** Interpreta el vector de resultados de la inferencia y selecciona la respuesta adecuada del chatbot.
- **Modelo de IA:** Desarrollado usando TensorFlow.js (o la biblioteca seleccionada). Está entrenado con un conjunto de datos sencillo y está optimizado para comprender preguntas simples en español.

Mantenimiento y Mejora

- **Entrenamiento adicional:** Para mejorar el modelo, se puede añadir más datos al conjunto de entrenamiento y ajustar los parámetros de la red neuronal.
- **Optimización:** Asegúrate de que el rendimiento sea adecuado, especialmente en dispositivos con recursos limitados.

Entrenamiento del Modelo en Google Colab

Se incluye un enlace a un archivo de Google Colab que contiene el código completo para entrenar el modelo utilizado en este proyecto. El notebook está diseñado para entrenar y exportar el modelo de inteligencia artificial que clasifica las intenciones del usuario.

Cómo acceder al Notebook

Puedes acceder y ejecutar el código del notebook directamente en Google Colab mediante el siguiente enlace:

[Entrenamiento](#)

Alojar el Chatbot en GitHub Pages

GitHub Pages es una excelente opción para alojar sitios web estáticos, y se puede utilizar para alojar el chatbot directamente desde un repositorio de GitHub. Sigue los siguientes pasos para desplegar el proyecto en GitHub Pages.

Pasos para Alojar el Chatbot en GitHub Pages

1. Crea un Repositorio en GitHub:

- Inicia sesión en tu cuenta de GitHub.
- Haz clic en el botón **New** para crear un nuevo repositorio.
- Asigna un nombre al repositorio (por ejemplo, "chatbot").
- Marca la opción **Public** para que el repositorio sea accesible a todos.
- Opcional: Agrega una descripción al repositorio.
- Haz clic en **Create repository**.

2. Sube los Archivos del Proyecto:

- En el nuevo repositorio, haz clic en el botón **Add file** y selecciona **Upload files**.
- Sube los siguientes archivos de tu proyecto:
 - `index.html`
 - `style.css`
 - `chatbot.js`
 - Los archivos de la carpeta `modelo/` (incluidos `model.onnx`, `intents.json`, etc.).
- Luego haz clic en **Commit changes** para guardar los archivos en el repositorio.

3. Configura GitHub Pages:

- Una vez que los archivos están subidos, ve a la pestaña **Settings** del repositorio.
- Desplázate hacia abajo hasta la sección **Pages**.
- En **Source**, selecciona la rama `main` (o `master` dependiendo del nombre de la rama principal de tu repositorio).
- En el menú desplegable que aparece al lado, selecciona la opción `/ (root)` para que el contenido del repositorio raíz se publique.
- Haz clic en **Save**.

4. Verifica el Despliegue:

- GitHub generará una URL donde se alojará tu chatbot (normalmente en el formato `https://<tu-usuario>.github.io/<nombre-del-repositorio>/`).
- Espera unos minutos mientras GitHub Pages despliega tu sitio.

- Una vez completado, abre el enlace para verificar que el chatbot está funcionando correctamente.

Ejemplo de URL Final:

Una vez que el despliegue esté completo, deberías poder acceder al chatbot desde una URL como la siguiente (dependiendo de tu nombre de usuario y el nombre del repositorio):

- <https://<tu-usuario>.github.io/chatbot/>